

Machine Learning: Comprehensive Theory and Concepts Study Guide

Module 1: Machine Learning Basics

1. Fundamentals

- **What is Artificial Intelligence (AI)?** AI is the broad field of computer science dedicated to building hardware and software systems capable of simulating human intelligence, reasoning, problem-solving, and decision-making.
- **What is Machine Learning (ML)?** A subset of AI focused on building algorithms that learn patterns directly from historical data to make predictions or decisions without being explicitly programmed.
- **What is Deep Learning (DL)?** A specialized subset of ML that uses multi-layered artificial neural networks (Deep Neural Networks) to automatically learn hierarchical feature representations from raw input data.
- **AI vs ML vs DL** AI is the overall envelope. ML is a subset inside AI that uses statistical learning algorithms. DL is a subset inside ML that uses multi-layer neural networks.
- **Why is Machine Learning important?** It enables automation of complex tasks (like image classification or speech translation) that are too complex for manual rule-based programming. It scales data analysis to identify non-linear relationships across millions of records.
- **Applications of Machine Learning**
 - Recommendation engines (Netflix, Amazon).
 - Autonomous vehicles and robotics.
 - Fraud detection and risk assessment.
 - Medical diagnostics and drug discovery.
 - Natural Language Processing (chatbots, translation).

2. Learning paradigms

- **Supervised Learning** The algorithm is trained on a labeled dataset containing inputs (X) and matching output labels (y). The model learns a mapping function $f(X) = y$.
- **Unsupervised Learning** The algorithm is trained on unlabeled data. It seeks to find hidden patterns, groupings, or structures within the data without human assistance.
- **Reinforcement Learning** An agent learns how to behave in an environment by performing actions and receiving feedback in the form of rewards or penalties. It aims to maximize cumulative rewards.

- **Semi-Supervised Learning** Combines a small volume of labeled data with a large volume of unlabeled data to train models, reducing labeling costs.
 - **Machine Learning Lifecycle**
 1. **Data Collection:** Gathering raw structured or unstructured data.
 2. **Data Cleaning:** Handling missing values, duplicate records, and outliers.
 3. **Feature Engineering:** Transforming, creating, scaling, and selecting input attributes.
 4. **Model Training:** Fitting ML algorithms to historical data.
 5. **Model Evaluation:** Testing model accuracy on validation/test subsets.
 6. **Hyperparameter Tuning:** Optimizing model configurations (e.g., learning rate).
 7. **Deployment:** Exporting the model to production APIs.
 8. **Monitoring & Retraining:** Tracking drift and retraining models as accuracy degrades.
-

Module 2: Data Preprocessing

- **Data Collection:** Pulling raw data from sources like databases, APIs, web scraping, or file streams.
- **Data Cleaning**
 - **Missing Values:** Can be dropped or imputed (using Mean/Median for numerical fields, Mode for categorical, or predictive models like KNN Imputer).
 - **Duplicate Data:** Identifying and removing duplicate rows using unique constraints.
 - **Outliers:** Extremes that deviate from other observations. Detected using Z-scores ($|Z| > 3$) or Interquartile Range (IQR):

$$\text{Outlier Range} < Q1 - 1.5 \cdot \text{IQR} \quad \text{or} \quad > Q3 + 1.5 \cdot \text{IQR}$$

Handled via trimming, log-transformations, or clipping (winsorization).

- **Data Transformation:** Converting features into forms suitable for ML algorithms (e.g., log transforms to fix skewed distributions).
- **Feature Engineering:** Creating new features or modifying existing ones (e.g., extracting "Day of Week" from a date string) to help the model learn better.
- **Feature Selection:** Filtering out redundant or noisy columns. Done via filter methods (Correlation, Chi-Square), wrapper methods (Recursive Feature Elimination), or embedded methods (Lasso/L1 regularization).
- **Categorical Encodings**
 - **Label Encoding:** Maps each category value to a unique integer (e.g. `['Red', 'Blue']` → `[0, 1]`). Use only for ordinal categories where hierarchy matters.
 - **One-Hot Encoding:** Creates dummy binary columns for each category. Prevents model from assuming ordinal relationships but increases feature dimensionality.
 - **Ordinal Encoding:** Maps ordered categories to sequential integers (e.g. `['Low', 'Medium', 'High']` → `[1, 2, 3]`).

- **Target Encoding:** Replaces each category value with the mean of the target variable for that category. Can lead to overfitting/data leakage if not regularized.
 - **Train-Test Split:** Splitting data (commonly 80/20) to train the model on one subset and evaluate it on a completely unseen test subset.
-

Module 3: Feature Scaling

- **Why Feature Scaling?** Features with large scales (e.g., Salary: \$100,000) will dominate features with small scales (e.g., Age: 25) during model optimization, leading to slow convergence or biased models.
- **Scaling Methods**
 - **Normalization (Min-Max Scaling):** Rescales values to a range between 0 and 1. Highly sensitive to outliers.

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

- **Standardization (Z-score Scaling):** Centers data around a mean of 0 with a standard deviation of 1.

$$X_{std} = \frac{X - \mu}{\sigma}$$

- **Robust Scaling:** Uses the Median and IQR, making it robust to outliers.

$$X_{robust} = \frac{X - \text{median}}{\text{IQR}}$$

- **When Scaling is Required?**
 - *Required for:* Distance-based algorithms (KNN, SVM, K-Means) and Gradient Descent optimization models (Linear/Logistic Regression, Neural Networks).
 - *Not required for:* Tree-based algorithms (Decision Trees, Random Forest, Gradient Boosting, XGBoost) because they split nodes on one feature at a time.
-

Module 4: Supervised Learning

- **Classification** Predicts a discrete categorical class label (e.g., predicting if an email is "Spam" or "Not Spam"). Output is a class or a probability distribution over classes.
- **Regression** Predicts a continuous numerical value (e.g., predicting a house price).
- **Classification vs Regression**

Feature	Classification	Regression
Output Type	Discrete labels / categories.	Continuous numerical values.
Objective	Find decision boundaries.	Fit a line/curve of best fit.
Metrics	Accuracy, Precision, Recall, F1, AUC.	MAE, MSE, RMSE, R ² Score.

Module 5: Regression Algorithms

- **Linear Regression** Models the relationship between a single independent variable and a dependent variable by fitting a straight line: $y = \beta_0 + \beta_1 x + \epsilon$.
- **Multiple Linear Regression** Fits a hyperplane to model a dependent variable using multiple independent variables: $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$.
- **Polynomial Regression** Models non-linear relationships by adding polynomial terms (e.g., x^2 , x^3) to the linear regression model.
- **Ridge Regression (L2 Regularization)** Linear regression that adds a squared magnitude of coefficients penalty to the loss function to prevent overfitting:

$$\text{Loss} = \text{OLS Loss} + \lambda \sum \beta_i^2$$

- **Lasso Regression (L1 Regularization)** Linear regression that adds an absolute magnitude of coefficients penalty to the loss function:

$$\text{Loss} = \text{OLS Loss} + \lambda \sum |\beta_i|$$

> [!TIP] > Lasso can shrink coefficients to exactly 0, serving as an automatic feature selection tool.

- **Elastic Net** Combines both L1 (Lasso) and L2 (Ridge) penalties in the loss function.
- **Assumptions of Linear Regression**
 1. **Linearity**: Relationship between inputs and output is linear.
 2. **Independence**: Residual errors are independent of each other.
 3. **Homoscedasticity**: The variance of residual errors is constant across all input levels.
 4. **Normality**: Residual errors are normally distributed.

Module 6: Classification Algorithms

- **Logistic Regression** Used for binary classification. Passes linear output through the **Sigmoid function** $p = 1/(1 + e^{-z})$ to map output values to probabilities between 0 and 1.
- **Decision Tree** Recursively splits data into nodes based on features that maximize node purity (measured using **Gini Impurity** or **Entropy/Information Gain**).

- **Random Forest** An ensemble bagging method that trains multiple independent decision trees on bootstrap datasets (sampling with replacement) and aggregates their predictions. Reduces overfitting.
 - **Naive Bayes** A probabilistic classifier based on Bayes' Theorem. It assumes "naive" conditional independence between all input features.
 - **KNN (K-Nearest Neighbors)** A non-parametric, instance-based classifier. Classifies a data point based on the majority label of its K closest neighbors using distance metrics (e.g., Euclidean).
 - **Support Vector Machine (SVM)** Finds the optimal hyperplane that separates classes with the maximum margin. Uses the **Kernel Trick** to project non-linear data into higher dimensions for linear separation.
 - **Ensemble Boosting Algorithms**
 - **AdaBoost**: Trains weak learners sequentially, increasing the weights of misclassified records in each step.
 - **Gradient Boosting**: Sequentially builds trees to predict the residual errors of preceding trees.
 - **XGBoost**: Highly optimized gradient boosting implementation using regularization, cache-aware block memory structures, and handling of sparse matrices.
 - **LightGBM**: Fast boosting framework using histogram-based algorithms and leaf-wise (vertical) tree growth.
 - **CatBoost**: Optimized framework designed to handle categorical features natively without manual encoding.
-

Module 7: Clustering

- **K-Means** An unsupervised algorithm that partitions data into K clusters. It iteratively assigns points to the nearest centroid and updates centroids to minimize the Within-Cluster Sum of Squares (WCSS).
 - *Elbow Method*: Plots WCSS vs. K to find the optimal cluster count at the "elbow" point.
 - **Hierarchical Clustering** Creates a tree-like structure of clusters (represented in a **Dendrogram**). Can be **Agglomerative** (bottom-up: starts with single points and merges them) or **Divisive** (top-down).
 - **DBSCAN (Density-Based Spatial Clustering of Applications with Noise)** Groups points based on density. Categorizes points as Core points, Border points, or Noise.
 - *Advantages*: Can identify clusters of arbitrary shapes and automatically detects outliers/noise without requiring the user to specify K .
-

Module 8: Dimensionality Reduction

- **PCA (Principal Component Analysis)** A linear dimensionality reduction technique. Projects data onto orthogonal axes (Principal Components) that maximize variance, reducing features while preserving information.
 - **LDA (Linear Discriminant Analysis)** A supervised dimensionality reduction and classification technique that projects features to maximize class separability.
 - **t-SNE (t-Distributed Stochastic Neighbor Embedding)** A non-linear dimensionality reduction technique. Excellent for visualizing high-dimensional datasets in 2D or 3D space by preserving local structures.
 - **UMAP (Uniform Manifold Approximation and Projection)** A fast non-linear dimensionality reduction technique based on manifold learning. It preserves global structures better and runs faster than t-SNE.
-

Module 9: Model Evaluation (Classification)

1. Confusion Matrix

A tabular report mapping prediction accuracy: * **True Positive (TP)**: Predicted Positive, Actual Positive. * **False Positive (FP)**: Predicted Positive, Actual Negative (Type I Error). * **True Negative (TN)**: Predicted Negative, Actual Negative. * **False Negative (FN)**: Predicted Negative, Actual Positive (Type II Error).

2. Metrics

- **Accuracy**: $(TP + TN) / (TP + TN + FP + FN)$. Can be misleading for imbalanced datasets.
- **Precision**: $TP / (TP + FP)$. Measures accuracy of positive predictions (minimizes False Positives).
- **Recall (Sensitivity)**: $TP / (TP + FN)$. Measures percentage of actual positives captured (minimizes False Negatives).
- **Specificity**: $TN / (TN + FP)$. Measures percentage of actual negatives captured.
- **F1 Score**: The harmonic mean of Precision and Recall:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **ROC Curve & AUC**
 - **ROC Curve**: Plots True Positive Rate (Recall) vs. False Positive Rate (1 – Specificity) across different probability classification thresholds.
 - **AUC (Area Under Curve)**: Measures the overall ability of a model to distinguish between classes. A perfect model has an AUC of 1.0.

- **Precision-Recall Curve** Plots Precision vs. Recall. Preferred over ROC-AUC for evaluating models trained on highly imbalanced datasets.
-

Module 10: Regression Metrics

- **MAE (Mean Absolute Error):** Average of absolute differences. Less sensitive to extreme outliers:

$$\text{MAE} = \frac{1}{n} \sum |y_i - \hat{y}_i|$$

- **MSE (Mean Squared Error):** Average of squared differences. Penalizes large errors heavily:

$$\text{MSE} = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

- **RMSE (Root Mean Squared Error):** Square root of MSE. Keeps error in the same units as the target variable:

$$\text{RMSE} = \sqrt{\text{MSE}}$$

- **R² Score (Coefficient of Determination):** The proportion of target variance explained by the model:

$$R^2 = 1 - \frac{\text{Sum of Squared Residuals (RSS)}}{\text{Total Sum of Squares (TSS)}}$$

- **Adjusted R²:** Penalizes the addition of irrelevant features that do not improve model fit:

$$\text{Adjusted } R^2 = 1 - \left[\frac{(1 - R^2)(n - 1)}{n - k - 1} \right]$$

(where n is sample size, k is feature count).

Module 11: Model Validation

- **Cross Validation** Splitting data into multiple subsets iteratively to validate performance, ensuring the model generalizes well to unseen data.
- **K-Fold Cross Validation** Splits the dataset into K equal folds. The model is trained K times, each time using $K - 1$ folds for training and 1 fold for validation. Results are averaged.
- **Stratified K-Fold** A variation of K-Fold where each fold maintains the same percentage of class labels as the complete dataset. Essential for classification on imbalanced datasets.
- **Leave-One-Out Cross Validation (LOOCV)** A extreme case of K-Fold where $K = N$ (sample size). The model is trained on $N - 1$ samples and validated on the single remaining sample, repeated N times. High computational cost.

Module 12: Hyperparameter Tuning

- **Hyperparameter** Configuration settings set before training begins (e.g., depth of a tree, learning rate). Unlike *parameters*, they are not learned during training.
 - **Tuning Strategies**
 - **Grid Search CV**: Evaluates every parameter combination in a defined grid. Guarantees finding the best configuration in the grid but is slow.
 - **Random Search CV**: Randomly samples configurations from a distribution. Faster and often finds optimal settings quickly.
 - **Bayesian Optimization**: Uses a probabilistic model to evaluate parameters, selecting the next set to evaluate based on past results to find the global optimum faster.
-

Module 13: Overfitting & Underfitting

- **Overfitting** Model learns noise in the training data, leading to low training error but high test error. Has **High Variance** and **Low Bias**.
- **Underfitting** Model is too simple to capture patterns, leading to high training and test errors. Has **Low Variance** and **High Bias**.
- **Bias-Variance Tradeoff** The goal is to find the sweet spot where both bias and variance are minimized, preventing both underfitting and overfitting.

Low Bias <-----> High Bias
High Var <-----> Low Var
(Overfitting) (Underfitting)

- **Data Leakage** When information from the test dataset is inadvertently leaked to the model during training. E.g., performing scaling on the entire dataset *before* performing train-test split.
-

Module 14: Regularization

Prevents overfitting by adding a penalty to the loss function. * **L1 Regularization (Lasso)**: Adds absolute value penalty of weights. Performs feature selection by driving minor weights to exactly 0. * **L2 Regularization (Ridge)**: Adds squared value penalty of weights. Shrinks weights close to 0, but never drives them to 0. * **Elastic Net**: Incorporates both L1 and L2 penalties:

$$\text{Penalty} = \alpha L_1 + (1 - \alpha) L_2$$

Module 15: Ensemble Learning

- **Ensemble Learning** Combining predictions from multiple base models to improve performance and stability.
 - **Bagging (Bootstrap Aggregating)** Trains base models in parallel on bootstrap subsets. Aggregates predictions (e.g., Random Forest). Focuses on reducing **variance**.
 - **Boosting** Trains models sequentially. Each new model focuses on correcting errors made by preceding models (e.g., Gradient Boosting, XGBoost). Focuses on reducing **bias**.
 - **Voting Classifier** Combines predictions of multiple models. Can be **Hard Voting** (majority rule) or **Soft Voting** (averages class probabilities).
 - **Stacking** Trains multiple base models, then trains a meta-model using the base models' predictions as inputs to make the final prediction.
-

Module 16: Imbalanced Data

- **Imbalanced Dataset:** When one class significantly outnumbers the other class in classification problems (e.g., fraud detection: 99% non-fraud, 1% fraud).
 - **SMOTE (Synthetic Minority Over-sampling Technique):** Generates synthetic minority class instances along the line segments joining K-nearest neighbors.
 - **Undersampling:** Randomly removing majority class instances. Can lose critical information.
 - **Oversampling:** Duplicating minority class instances. Can lead to overfitting.
 - **Class Weight:** Adjusts the loss function to penalize misclassifying the minority class more heavily.
-

Module 17: Probability & Statistics

- **Mean:** Arithmetic average.
- **Median:** The middle value of a sorted distribution. Robust to outliers.
- **Mode:** The most frequent value.
- **Variance (σ^2):** Average squared deviation from the mean:

$$\text{Var} = \frac{\sum (x_i - \mu)^2}{N}$$

- **Standard Deviation (σ):** Square root of variance.
- **Covariance:** Measures how two variables change together.
- **Correlation:** Standardized covariance ranging between -1 and 1. Measures linear relationships (Pearson) or monotonic relationships (Spearman).
- **Bayes' Theorem:**

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

- **Normal Distribution:** Bell-shaped curve governed by mean (μ) and standard deviation (σ). Follows the **68-95-99.7 Empirical Rule**.
 - **Central Limit Theorem (CLT):** The sampling distribution of the sample mean approaches a normal distribution as sample size increases ($n \geq 30$), regardless of the shape of the population distribution.
-

Module 18: Time Series

- **Time Series:** Sequential data points collected over successive time intervals.
 - **Components:**
 - **Trend:** Long-term upward or downward movement.
 - **Seasonality:** Repeating fluctuations observed at regular intervals (e.g., weekly, yearly).
 - **Stationarity:** Statistical properties (mean, variance) do not change over time. Checked using the **Augmented Dickey-Fuller (ADF) test**. Models require stationary data to perform forecasts.
 - **ARIMA (Autoregressive Integrated Moving Average):** Standard forecasting model parameterized by (p, d, q) representing Auto-Regressive, Integrated (differencing to achieve stationarity), and Moving Average terms.
 - **SARIMA:** Adds seasonal terms $(P, D, Q)_s$ to ARIMA.
 - **Prophet:** An additive regression model open-sourced by Facebook, designed to handle seasonality, holidays, and trend shifts.
 - **Time Series Cross Validation:** Uses a rolling window or expanding training window (Time Series Split) to evaluate models, preventing future data leaks.
-

Module 19: Deep Learning Basics

- **Neural Network:** A computing model inspired by biological brains, containing Input, Hidden, and Output layers.
- **Perceptron:** The simplest single-layer neural network unit:

$$y = \text{Activation}(\sum w_i x_i + b)$$

- **Activation Functions**
 - **Sigmoid:** Maps output to range $(0, 1)$. Used for binary classification. Can trigger vanishing gradients.

- **ReLU (Rectified Linear Unit):** $f(x) = \max(0, x)$. Standard activation for hidden layers. Prevents vanishing gradients.
 - **Tanh:** Maps output to range $(-1, 1)$. Zero-centered.
 - **Softmax:** Converts vector outputs into probability distributions. Used in multi-class output layers.
 - **Optimization Components**
 - **Loss Function:** Measures difference between actual and predicted outputs (e.g., Binary Cross-Entropy, MSE).
 - **Optimizer:** Updates weights to minimize loss (e.g., SGD, Adam, RMSprop).
 - **Gradient Descent:** Optimization algorithm that iteratively updates weights in the direction of steepest descent of the loss function.
 - **Backpropagation:** Computes the gradients of the loss function with respect to weights using the chain rule of calculus, propagating errors backward.
 - **Epoch:** One full pass through the entire training dataset.
 - **Batch Size:** Number of training samples processed before updating weights.
 - **Learning Rate:** Step size taken during gradient descent optimization.
-

Module 20: CNN (Convolutional Neural Network)

Designed for processing spatial grid data like images. * **Convolution Layer:** Applies kernel filters across input matrices to extract feature maps (edges, patterns). Uses hyperparameters like kernel size, stride, and padding. * **Pooling Layer:** Reduces spatial dimensions (downsampling) to lower computational cost. Usually uses **Max Pooling** or **Average Pooling**. * **Flatten Layer:** Converts 2D feature maps into a 1D vector. * **Fully Connected (Dense) Layer:** Classifies features extracted by previous layers.

Module 21: RNN (Recurrent Neural Network)

Designed for processing sequential or time-series data. * **RNN:** Uses loop feedback mechanisms to retain memory of past inputs. * **LSTM (Long Short-Term Memory):** Solves long-term dependency limits using three gates: * **Forget Gate:** Decides what information to discard from cell state. * **Input Gate:** Decides what new information to store. * **Output Gate:** Decides what to output as hidden state. * **GRU (Gated Recurrent Unit):** A simplified LSTM using only two gates: **Reset Gate** and **Update Gate**. * **Vanishing / Exploding Gradients** Occurs when gradients shrink or grow exponentially during backpropagation in deep networks, preventing weights from updating. Fixed using ReLU, gradient clipping, residual connections, or LSTM/GRU networks.

Module 22: Autoencoders

- **Autoencoder** An unsupervised neural network designed to learn efficient data codings (dimensionality reduction).
 - **Encoder:** Compresses input data (X) into a lower-dimensional latent-space representation (z).
 - **Decoder:** Reconstructs the original data ($\hat{X}$) from the latent space representation (z).
 - **Reconstruction Error:** The loss function (e.g., MSE) measuring the difference between input X and reconstructed output $\hat{X}$.
-

Module 23: Deployment

- **Model Serialization** Saving trained models to disk.
 - **Pickle:** Standard Python serialization library.
 - **Joblib:** Optimized version of Pickle designed for serializing models with large NumPy arrays (e.g., Random Forest).
 - **FastAPI Deployment:** Wrapping a model inside a FastAPI REST framework to serve predictions via endpoint requests (e.g., POST `/predict`).
 - **Streamlit Deployment:** Creating simple, interactive Python-based web applications to demonstrate model predictions visually.
 - **Docker Deployment:** Containerizing the model, code, dependencies, and system settings inside a Docker container using a `Dockerfile`, ensuring reproducible executions in any environment.
-

Module 24: MLOps

- **CI/CD:** Automating building, testing, and deployment processes for code and ML models.
- **ML Pipeline:** Automating sequential steps from data ingestion, preprocessing, validation, training, to model deployment.
- **Model Monitoring**
 - **Data Drift:** Input data distribution changes in production compared to training (e.g. demographic shifts). Checked using Kolmogorov-Smirnov statistical tests.
 - **Concept Drift:** The relationship between input features and target labels changes (e.g. purchasing habits shift after economic changes).
- **Model Retraining:** Triggering pipeline runs periodically or automatically when model accuracy drops below a defined threshold.
- **Experiment Tracking:** Logging hyperparameter metrics, code versions, and artifact files using systems like **MLflow** or **Weights & Biases**.

Module 25: Production ML

- **Feature Store** A centralized directory that registers, stores, and serves standardized feature definitions for both offline training and online real-time inference (e.g., Feast).
 - **Batch Inference (Offline)** Running predictions on large volumes of data in batches periodically (e.g., overnight recommendation generation). High throughput, high latency.
 - **Real-Time Inference (Online)** Generating predictions on-demand for single requests instantly (e.g., real-time fraud checks). Low latency, high computational cost.
 - **Online Learning:** Model updates its weights incrementally as new individual data points arrive.
 - **Offline Learning:** Traditional model training on static batch datasets.
-

Module 26: Scenario-Based Questions

1. How do you choose an ML algorithm?

Choose based on the task (Regression vs. Classification), dataset size, interpretability requirements (Decision Trees are interpretable, Deep Learning is black-box), training speed limits, and performance goals. * *Small data / interpretable*: Linear/Logistic Regression, Decision Trees. * *Complex / non-linear*: Random Forest, XGBoost, Support Vector Machines. * *Images/Text*: Deep Learning (CNNs, Transformers).

2. How do you improve model accuracy?

Collect more training data, perform feature engineering, handle outliers and missing values, try ensemble methods (XGBoost, Stacking), and perform hyperparameter tuning.

3. How do you reduce overfitting?

Use regularization (L1/L2), collect more data, simplify model complexity, use dropout (in neural networks), apply cross-validation, or use ensemble bagging methods.

4. How do you handle missing values?

Impute values using Mean/Median (numerical) or Mode (categorical). For structured patterns, use iterative predictive imputation (KNN Imputer). If the missingness itself contains information, create a binary indicator column before imputing.

5. How do you handle outliers?

Detect outliers using IQR or Z-score checks. Handle them by dropping them (if they are measurement errors), clipping values (Winsorization), or applying log transforms to compress

values.

6. How do you handle categorical data?

Use One-Hot Encoding for low-cardinality nominal features, Ordinal Encoding for features with ordered structures, and Target Encoding or frequency encoding for high-cardinality nominal features.

7. How do you select features?

Use correlation matrices to identify and drop collinear columns. Use wrapper methods (Recursive Feature Elimination) or embedded feature penalties (Lasso regularization).

8. How do you evaluate a model?

Split data into train/test subsets. Use Stratified K-Fold cross-validation to assess generalization. Match metrics to business objectives (e.g., focus on Recall for medical diagnostics, Precision for spam detection).

9. Why Cross-Validation?

It ensures that model evaluation does not depend on a single, lucky train-test split, providing a reliable estimate of how the model performs on unseen data.

10. How do you deploy an ML model?

Serialize the model to a file using Joblib, wrap it inside a FastAPI app, build a Docker image containing the code and environment, and deploy the container to cloud container services (AWS ECS, Kubernetes).

11. What if production accuracy decreases?

Check for Data Drift and Concept Drift. Inspect input data distributions and compare them to training data. Trigger the retraining pipeline with recent labels.

12. How do you detect Data Drift?

Compare the distribution of features in production to the baseline training dataset using statistical checks like the Kolmogorov-Smirnov test (numerical) or Population Stability Index (PSI).

13. How do you retrain a model?

Automate the process using an orchestration tool (Airflow, Kubeflow) to execute data pull, preprocess, train, evaluate, and deploy steps. Retraining can be scheduled weekly, monthly, or triggered by drift detection alerts.

14. How do you monitor a deployed model?

Track model latency, system resources (CPU/RAM), data schema validation failures, predictive drift metrics, and business KPIs (e.g. CTR shifts).

15. How do you explain your ML project architecture?

Explain the pipeline using the data flow:



```
[Data Source] → [ETL Engine] → [Feature Store] → [Training Pipeline] → [Registry] → [API Se]
```

Module 27: Project-Based Questions (Retail Sales Forecasting)

- **Explain your Retail Sales Forecasting project.** Developed a forecasting pipeline to predict weekly store sales across retail branches. Handled structured historical sales logs, promotions data, and store profiles.
- **Why did you choose that algorithm?** Used **XGBoost Regressor** and **SARIMA** ensemble methods. XGBoost was chosen because it handles tabular non-linear relations and missing features efficiently. SARIMA captured explicit seasonal trends.
- **How did you perform feature engineering?**
 - Extracted temporal features: Day of week, Month, Week of Year, and Holiday flags.
 - Generated Lag Features (sales from 1, 2, and 4 weeks prior) and rolling averages to capture sales trends.
 - Target-encoded store and department IDs.
- **How did you evaluate the model?** Evaluated using Time Series Split (expanding window cross-validation) to prevent future data leakage.
- **Which evaluation metrics did you use?** **RMSE** to penalize large forecasting errors, and **MAPE** (Mean Absolute Percentage Error) to communicate performance to business stakeholders.
- **What challenges did you face?**
 - *Data Leakage*: Unintentionally using future promotional info in historical training folds. Fixed by setting up tight rolling pipelines.
 - *Data Skew*: Extreme sales peaks during holiday seasons. Solved by introducing specific holiday weight parameters.
- **How did you deploy the model?** Packaged the model into a Docker container serving predictions via FastAPI. Run on a cron schedule to generate weekly batch forecasts.
- **How would you improve and scale the solution?** Incorporate external features like local weather data or local economic indices. To scale, rebuild feature engineering pipelines using PySpark to handle 100+ million records.

★ Top 40 Most Frequently Asked ML Questions (Accenture)

1. **What is Machine Learning?** An AI discipline focused on building algorithms that learn patterns from data to make predictions.
2. **AI vs ML vs DL?** AI simulates human intelligence; ML is a data-driven subset of AI; DL is a subset of ML using deep neural networks.
3. **Types of Machine Learning?** Supervised (labeled data), Unsupervised (unlabeled), Reinforcement (rewards/penalties), Semi-Supervised (mixed).
4. **ML Lifecycle?** Data collection, cleaning, feature engineering, training, validation, tuning, deployment, monitoring.
5. **Supervised vs Unsupervised Learning?** Supervised maps inputs to outputs using labeled data. Unsupervised finds hidden structures in unlabeled data.
6. **Classification vs Regression?** Classification predicts discrete categories. Regression predicts continuous numerical outputs.
7. **What is Linear Regression?** Fits a straight line $y = mx + c$ to model continuous values, minimizing squared errors.
8. **What is Logistic Regression?** A classification algorithm that maps linear combinations of features to a probability between 0 and 1 using the Sigmoid function.
9. **What is a Decision Tree?** A model that splits data recursively at nodes based on features that maximize target purity.
10. **What is Random Forest?** An ensemble bagging algorithm that trains multiple independent decision trees and averages their outputs.
11. **What is XGBoost?** An optimized gradient boosting library implementing parallel tree construction, regularization, and handling of sparse matrices.
12. **Gradient Boosting vs Random Forest?** Random Forest builds trees in parallel (bagging) to reduce variance. Gradient Boosting builds trees sequentially to reduce bias by predicting prior errors.
13. **What is KNN?** An instance-based classifier that labels a new data point based on the majority label of its K closest neighbors.
14. **What is SVM?** A classifier that finds the separating hyperplane with the maximum margin between classes, using kernels for non-linear data.
15. **What is Naive Bayes?** A probabilistic classifier based on Bayes' Theorem, assuming conditional independence between all input features.
16. **What is K-Means?** An unsupervised algorithm partitioning data into K clusters by minimizing WCSS distance to centroids.
17. **What is PCA?** A linear dimensionality reduction technique projecting data onto principal component axes that capture the maximum variance.
18. **What is Feature Engineering?** Modifying or creating features from raw data to help ML algorithms learn patterns more effectively.

19. **What is Feature Selection?** Identifying and retaining only the most informative features, dropping redundant or noisy columns.
20. **One-Hot Encoding vs Label Encoding?** One-Hot Encoding creates binary columns for each category (for nominal data). Label Encoding maps categories to sequential integers (for ordinal data).
21. **How do you handle Missing Values?** Through removal (if sparse) or imputation using mean/median/mode or predictive models (KNN Imputer).
22. **Why Train-Test Split?** To evaluate model performance on unseen data, checking for overfitting.
23. **What is Cross-Validation?** Splitting data into K folds to train and validate iteratively, providing a robust estimate of performance.
24. **Grid Search vs Random Search?** Grid Search evaluates every combination in a grid. Random Search randomly samples combinations, running faster.
25. **What is a Confusion Matrix?** A table summarizing model predictions: TP, FP, TN, FN.
26. **What is Accuracy?** The proportion of correct predictions out of all predictions. Can be misleading on imbalanced data.
27. **What is Precision?** The ratio of true positive predictions to all positive predictions: $TP/(TP + FP)$.
28. **What is Recall?** The ratio of true positives to actual positives: $TP/(TP + FN)$.
29. **What is F1 Score?** The harmonic mean of Precision and Recall, useful for evaluating models on imbalanced datasets.
30. **What is ROC-AUC?** ROC plots TPR vs. FPR across thresholds. AUC measures the model's overall ability to distinguish between classes.
31. **What is MAE?** Mean Absolute Error. The average of absolute residuals, robust to outliers.
32. **What is RMSE?** Root Mean Squared Error. Penalizes large errors heavily, keeping the error unit consistent with the target.
33. **What is R² Score?** The proportion of variance in the target variable explained by the features.
34. **What is Overfitting?** When a model learns training data noise, failing to generalize to unseen test data.
35. **What is Underfitting?** When a model is too simple to capture patterns in the training data.
36. **Bias-Variance Tradeoff?** Balancing model complexity to minimize errors from both bias (underfitting) and variance (overfitting).
37. **What is Regularization (L1/L2)?** Adding absolute (L1) or squared (L2) coefficient penalties to the loss function to prevent overfitting.
38. **Bagging vs Boosting?** Bagging trains models in parallel to reduce variance. Boosting trains models sequentially to reduce bias.
39. **Data Drift vs Concept Drift?** Data Drift is a change in the input data distribution. Concept Drift is a change in the relationship between input features and target labels.
40. **How do you handle Imbalanced Data?** Using SMOTE oversampling, majority class undersampling, adjusting class weights, or using Precision-Recall curves instead of ROC-

AUC.



Module Summary & Key Takeaways

Theory Focus: Feature Engineering, Bias-Variance Tradeoff, Regularization, and Evaluation Metrics.

This module covers the core mathematical and procedural foundations of machine learning pipelines. Key takeaways include:

- **Preprocessing & Data Leakage:** Raw features must be scaled (standardized or normalized) and imputed carefully. Fitting preprocessing transformers must be performed strictly on the training partition to prevent lookahead data leakage.
- **Bias-Variance Tradeoff:** Models must balance training fit (bias) with generalizability to unseen data (variance). Regularization manages overfitting by adding penalty terms to the loss function: **L1 Regularization (Lasso)** performs feature selection by shrinking parameters to absolute zero, while **L2 Regularization (Ridge)** penalizes high weight magnitudes.
- **Rigorous Evaluation:** Predictive outputs must be evaluated using robust metrics. Classifiers use Precision, Recall, F1-Score, and ROC-AUC (especially for imbalanced data), while regressors utilize MAE, RMSE, and the R^2 score to measure explained variance.